전기전자통신공학부 정보통신공학

머신러닝 기말 프로젝트 보고서

실증적 문제에 대한 딥러닝 기반의 알고리즘 설계 및 검증

- 키보드 타건 및 마우스 소리 분석을 통한 활동 내용 분류 -

이름 : 류황민

과목/분반 : 머신러닝 / 1분반

제 출 일 : 2025.12.10



목 차

Ⅰ. 서론 ····· 01

1. 문제 정의 ····· 01
2. 선정 배경 및 필요성 ····· 01
3. 기존 연구와의 차별성 ····· 01
4. 데이터 수집 방법 ····· 01
5. 데이터 특성 분석 및 전처리 필요성 ····· 02

Ⅱ. 본론 ····· 03

1. 데이터 정규화 및 전처리 ····· 03
   1.1) 원본 오디오 정리 및 신호 안정화 ····· 03
   1.2) 세그먼트 분할 및 클래스별 hop 조정 ····· 03
   1.3) Log-Mel Spectrogram 기반 특징 추출 ····· 04
      1.3.1) 로그-멜 스펙트로그램 선택 배경 ····· 05
   1.4) 데이터 분할 및 데이터 누수 방지 전략 ····· 05
   1.5) 전처리 결과 저장 및 관리 ····· 06
   1.6) 전처리 결과 시각화 ····· 07
2. 모델 구조 및 설계 ····· 08
   2.1) 초기 모델 구조 ····· 09
   2.2) 개선된 모델 구조 ····· 10
3. 단계별 실험 설계 및 분석 ····· 13
   3.1) 첫 번째 실험 ····· 13
   3.2) 두 번째 실험 ····· 14
   3.3) 세 번째 실험 ····· 15
   3.4) 네 번째 실험 ····· 16
   3.5) 다섯 번째 실험 ····· 18
   3.6) 마지막 실험 ····· 19
4. 최종 모델 성능 평가 ····· 20
5. 한계점 및 향후 개선 방향 ····· 21

Ⅲ. 결론 ····· 22

1. 고찰 ····· 22

참고자료 ····· 23

Ⅰ. 서 론

1. 문제정의

컴퓨터 사용 환경에서 사용자는 문서 작성, 웹서핑, 게임, 채팅과 같이 다양한 활동을 진행한다. 하지만 이러한 활동들은 모두 키보드와 마우스라는 동일한 입력 장치를 사용하기 때문에, 외부에서 들리는 소리만으로 사용자가 어떤 활동을 하고 있는지 구별하기 어렵다. 본 프로젝트는 10초 길이의 키보드·마우스 입력음을 기반으로 사용자의 활동을 Writing, Web Surfing, Game, Chatting 네 가지 중 하나로 분류하는 딥러닝 모델을 제작하는 것을 목표로 한다.

2. 선정 배경 및 필요성

학생이나 직장인이 작업을 할 때 웹서핑이나 게임처럼 의도하지 않은 활동으로 주의가 흐트러지는 경우가 많다. 만약 사용자의 입력 패턴을 실시간으로 분석해 현재 작업이 목표 작업과 다를 경우 알려주는 시스템이 있다면, 스스로 작업 흐름을 관리하는 데 도움이 될 수 있다.

이번 프로젝트는 이러한 활용 가능성을 염두에 두고 있으며, 키보드·마우스 입력음만으로도 활동 분류가 가능한지 실험적으로 확인하는 데 목적이 있다. 음성 데이터를 포함하지 않기 때문에 개인정보 침해 위험이 낮다는 점도 장점이다.

3. 기존 연구와의 차별성

기존 오디오 분류 연구는 대부분 음성, 악기 소리, 환경 소음처럼 서로 명확히 구분되는 음향 신호를 다룬다. 그러나 본 프로젝트는 짧고 불규칙적인 비음성 입력음을 대상으로 하며, 특히 Chatting과 Writing처럼 구별이 어렵고 유사한 입력 패턴을 가진 활동까지 포함한다.

또한 키보드뿐 아니라 마우스 클릭, 스크롤 소리도 함께 포함해 다양한 입력 조건을 반영했으며, 클래스 간 데이터 편차를 완화하기 위해 hop 크기를 조정하고 환경을 달리해 데이터를 수집했다는 점에서 기존 연구와 차별성을 가진다.

4. 데이터 수집 방법

데이터는 직접 녹음과 지인 협력을 통해 수집하였다. 인위적으로 통제된 실험실 환경이 아닌, 실제 컴퓨터 사용 환경에서 문서 작성, 웹서핑, 게임, 채팅 등의 활동을 수행하면서 자연스럽게 발생하는 키보드와 마우스 입력음을 녹음하였다. 다만 불필요한 변수를 줄이기 위해 주변 대화 소리나 음악 등 명확한 배경 소음은 최대한 배제한 상태에서 녹음을 진행하였다.

활동 종류에 따라 Chatting, Game, Web Surfing, Writing 디렉터리로 구분하여 정리하였다. 또한 서로 다른 두 종류의 키보드와 마우스 환경에서 데이터를 확보하여, 입력 장치 차이에

따른 패턴 변화를 일부 반영할 수 있도록 하였다.

초기 데이터셋 구성 단계에서는 문서 작성, 웹서핑, 게임의 세 가지 활동만을 포함하였다. 그러나 실험 과정에서 채팅 입력의 짧고 반복적인 타건 패턴이 다른 활동과 구별되는 특성을 보인다는 점을 확인하였고, 이를 분류 대상으로 포함하는 것이 모델 성능 향상에 도움이 될 것으로 판단하였다. 이에 따라 최종 데이터셋은 Chatting, Game, Web Surfing, Writing의 네 개 클래스로 확장하였다.

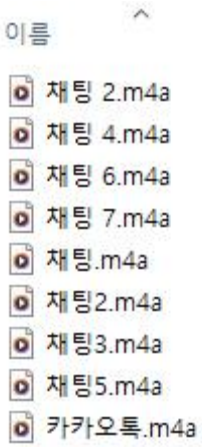


그림 1



그림 2

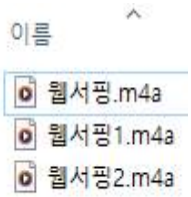


그림 3

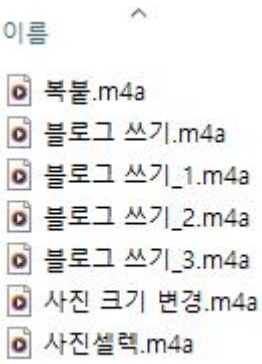


그림 4

그림 1부터 그림 4는 각 활동별로 직접 녹음한 원본 오디오 파일의 예시를 보여준다. 그림 1은 채팅 입력, 그림 2는 게임 플레이 중 입력, 그림 3은 웹서핑 중 입력, 그림 4는 문서 작성 중 입력에 해당하는 녹음 파일이다. 각 데이터셋의 수집 시간은 Chatting 약 30분, Game 약 4시간, Web Surfing 약 30분, Writing 약 3시간 수준으로 구성되었다. 이처럼 클래스별 데이터 수집 시간에는 차이가 존재하며, 이는 이후 전처리 및 학습 단계에서 데이터 불균형을 고려해야 하는 요인으로 작용하였다.

5. 데이터 특성 분석 및 전처리 필요성

녹음 장치가 다르고 녹음 환경도 일정하지 않아 원본 데이터는 샘플레이트, 채널 수 등이 모두 달랐다. 또한 클래스별 입력 습관의 차이 때문에 데이터 양과 품질도 균일하지 않았다. 이러한 데이터는 그대로 모델에 사용하면 잡음에 영향받거나 특정 클래스에 편향된 학습이 발생하는 문제가 생길 수 있다.

또한 같은 원본 파일에서 나온 여러 세그먼트가 학습·검증·테스트에 모두 섞여버리면, 모델이 실제 활동 패턴이 아니라 특정 환경의 잡음을 학습하는 데이터 누수가 발생할 수 있다고 한다. 따라서 파일 단위 분리, 샘플레이트 통일, 특징 추출 방식 정규화 등이 필요했고, 이 과정은 모델 성능의 신뢰도를 높이는 핵심 기반이 되었다.

Ⅱ. 본 론

1. 데이터 정규화 및 전처리 방법

본 프로젝트에서는 다양한 환경에서 수집된 키보드·마우스 입력음을 딥러닝 모델이 학습하기 적합한 수치 형태로 변환하는 것을 목표로 전처리 파이프라인을 구성하였다. 전체 전처리 과정은 원본 오디오 정리, 세그먼트 분할, 특징 추출, 데이터 분할 및 정규화의 단계로 이루어진다.

1.1) 원본 오디오 정리 및 신호 안정화

수집된 원본 오디오는 실제 작업 환경에서 직접 녹음된 파일로, 녹음 장치와 환경에 따른 신호 차이가 존재할 수 있다. 이를 최소화하기 위해 모든 오디오는 WAV 형식으로 통일한 뒤, 샘플레이트를 16kHz로 재조정하고 모노 채널로 변환하였다. 이 과정은 서로 다른 녹음 환경에서 발생하는 주파수 해상도 차이와 채널 간 편차를 줄이기 위한 것이다.

이후 간단한 고역 통과 필터를 적용하여 DC 오프셋을 제거하였다. DC 오프셋은 오디오 신호의 평균값이 0이 아닌 경우 발생하는 현상으로, 주로 녹음 장치나 환경에 의해 발생한다. 본 전처리에서는 인접 샘플 간 차분을 이용한 간단한 방식으로 DC 성분을 제거하여, 이후 주파수 분석 단계에서 불필요한 저주파 성분의 영향을 줄였다.

그림 1은 하나의 원본 오디오 파일에 대해 시간 축에 따른 파형을 시각화한 결과로, 키보드 입력과 마우스 클릭이 반복적으로 발생하는 입력음의 특성을 확인할 수 있다.

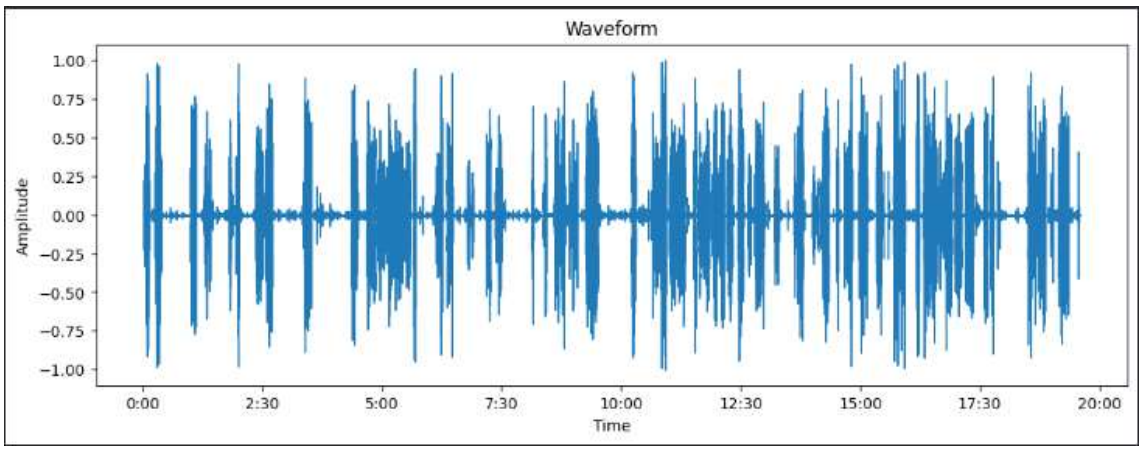


그림 1

1.2) 세그먼트 분할 및 클래스별 hop 조정

정제된 오디오 신호는 일정한 길이의 분석 단위를 만들기 위해 10초 길이의 세그먼트로 분할하였다. 세그먼트 분할에는 슬라이딩 윈도우 방식을 사용하였으며, 분할 간격인 hop 크기는 클래스별 데이터 양을 고려하여 다르게 설정하였다.

Chatting과 Web Surfing 클래스는 상대적으로 수집된 데이터 양이 적기 때문에, 초기 실험 단계에서는 hop 크기를 짧게 설정하여 세그먼트 수를 충분히 확보하고자 하였다. 반면 Game과 Writing 클래스는 데이터 양이 많아, 중복 세그먼트가 과도하게 생성되는 것을 방지하기 위해 hop 크기를 상대적으로 크게 설정하였다.

다만 hop 크기는 하나의 고정된 전처리 설정으로 두지 않고, 각 실험 단계마다 서로 다른 hop 조합을 적용하여 성능 변화를 비교 분석하였다. 초기 실험에서는 1초와 3초 조합을 사용하였고, 이후 실험에서는 0.5초, 1.5초, 3초 등 다양한 hop 값을 적용하여 세그먼트 중복도와 분류 성능 간의 관계를 확인하였다. 이러한 반복 실험을 통해 hop 크기 조정이 성능에 미치는 영향을 분석하고자 하였으며, 클래스 간 데이터 불균형을 완화하는 데 목적을 두었다.

1.3) Log-Mel Spectrogram 기반 특징 추출

분할된 각 세그먼트는 합성곱 신경망에 입력하기 위해 Log-Mel Spectrogram 형태의 특징으로 변환하였다. 먼저 짧은 시간 구간에 대해 단기 푸리에 변환을 수행한 뒤, 주파수 축을 멜 스케일로 변환하여 사람의 청각 특성을 반영하였다. 이후 로그 스케일을 적용하여 에너지 차이를 완화하고, 입력 값의 분포를 안정화하였다.

본 프로젝트에서는 멜 필터 개수를 64개로 설정하였으며, 이는 최종적으로 생성되는 스펙트로그램의 주파수 축 해상도가 64가 됨을 의미한다. 이 값은 모델 입력 크기와 일관성을 유지하면서도 키보드·마우스 입력과 같은 비음성 이벤트의 주파수 패턴을 표현하기에 충분한 해상도라고 판단하였다.

그림 2은 그림 1의 동일한 오디오 파일에 대해 Log-Mel Spectrogram으로 변환한 결과로, 시간에 따라 반복적으로 나타나는 입력 이벤트가 주파수 영역에서 뚜렷한 수직 패턴으로 나타나는 것을 확인할 수 있다.

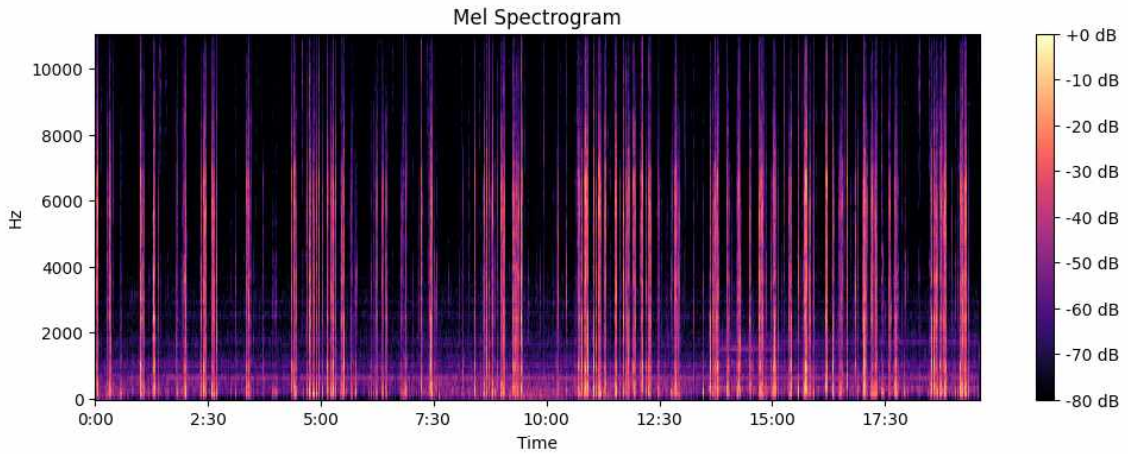


그림 2

1.3.1) 로그-멜 스펙트로그램 선택 배경

초기 단계에서는 음성 파일 자체를 그대로 입력으로 사용해 모델을 학습시키는 방안을 가장 먼저 떠올렸다. 그러나 실제로 구현을 시작하려 하자, 음성 신호를 어떤 형태로 모델에 입력해야 하는지조차 명확하지 않아 상당한 막막함을 느꼈다. 단순한 수치 배열로 음성을 입력하는 것이 과연 의미 있는 학습으로 이어질 수 있는지 확신하기 어려웠고, 시간에 따라 변화하는 파형과 주파수 성분을 동시에 고려해야 한다는 점에서 접근 방법을 잡기 힘들었다.

이러한 고민 속에서 다양한 방법을 고민해보던 도중 음성 신호를 이미지 형태로 변환하여 다루는 방법이 존재한다는 점을 알게 되었다. 소리를 시각적인 정보로 변환하면, 수업에서 배운 합성곱 신경망 구조를 그대로 활용할 수 있고, 모델 구조 설계와 실험 과정에 더 집중할 수 있을 것이라 생각하였다. 이후 관련 자료를 탐색하던 중, 멜 스펙트로그램이라는 개념을 접하게 되었고, 음성의 시간 변화와 주파수 특성을 동시에 표현할 수 있다는 점이 흥미롭게 느껴졌다. 특히 소리를 사람이 인지하는 방식과 유사하게 표현하려는 시도라는 설명이 본 프로젝트의 방향성과 잘 맞는다고 판단하였다.

여러 블로그와 자료를 살펴보며, 멜 스펙트로그램에 로그 스케일을 적용한 로그 멜 스펙트로그램이 소리의 강약 차이를 보다 안정적으로 표현할 수 있다는 점을 이해하게 되었다. 이 과정에서 "음성을 그대로 다루기보다, 사람이 소리를 인식하는 구조를 흉내 낸 숫자 배열로 변환해 학습시키는 방법도 있구나"라는 관점의 전환이 이루어졌다. 이는 음성 처리라는 낯선 문제를 이미지 기반 합성곱 신경망 모델로 접근할 수 있는 좋은 방법이라고 느껴졌다.

결과적으로 로그 멜 스펙트로그램은 음성 신호를 시각적 패턴으로 변환함으로써, 복잡한 음향 신호를 보다 직관적인 형태로 다룰 수 있게 해주었고, 수업에서 학습한 CNN 구조를 그대로 활용할 수 있는 기반을 제공하였다. 이러한 이유로 본 프로젝트에서는 로그 멜 스펙트로그램을 핵심 입력 특징으로 선택하게 되었다.

1.4) 데이터 분할 및 데이터 누수 방지 전략

로그 멜 스펙트로그램 생성 과정에서는 하나의 원본 음성 파일을 일정 길이의 세그먼트로 분할한 뒤, hop 단위로 슬라이딩하며 다수의 학습 데이터를 생성하였다. 이 방식은 제한된 녹음 분량에서도 충분한 학습 샘플을 확보할 수 있다는 장점이 있지만, 동일한 원본 파일에서 생성된 세그먼트들이 매우 유사한 특성을 갖는다는 점에서 데이터 분할 방식에 대한 추가적인 고려가 필요하다고 판단하였다.

특히 같은 파일에서 연속적으로 추출된 세그먼트들은 시간적으로 인접해 있으며, 주파수 분포 또한 거의 유사하다. 만약 이러한 세그먼트들이 무작위로 섞여 학습 데이터와 검증 또는 테스트 데이터에 동시에 포함될 경우, 모델은 사실상 매우 비슷한 입력을 학습 단계와 평가 단계에서 모두 접하게 된다. 이 경우 모델이 실제로는 보지 못한 데이터에 대해 일반화 성능을 보이는 것처럼 보일 수 있으며, 평가 정확도가 실제 성능보다 과도하게 높게 측정되는 데이터 누수 문제가 발생할 수 있다.

이러한 문제를 방지하기 위해 본 프로젝트에서는 세그먼트 단위가 아닌 원본 파일 단위로 데이터셋을 분할하는 방식을 채택하였다. 전처리 코드에서는 GroupShuffleSplit을 사용하여 각 음성 파일의 경로를 하나의 그룹으로 지정하고, 동일한 파일에서 생성된 모든 세그먼트가 반드시 train, validation, test 중 하나의 데이터셋에만 포함되도록 설계하였다. 이를 통해 학습 과정에서 사용된 입력이 평가 단계에 다시 등장하는 상황을 원천적으로 차단하였다.

데이터 분할 비율은 전체 파일 기준으로 train 약 60퍼센트, validation 약 20퍼센트, test 약 20퍼센트가 되도록 설정하였다. 먼저 전체 파일을 대상으로 train과 test를 분리한 뒤, train에 해당하는 파일들을 다시 나누어 validation 세트를 구성하는 단계적 분할 방식을 적용하였다. 이러한 절차는 검증 및 테스트 데이터가 학습 과정에 직간접적으로 영향을 받지 않도록 하기 위한 목적에서 설계되었다.

이와 같은 데이터 분할 전략을 통해 모델이 특정 녹음 환경이나 입력 장치의 고유한 특성을 외워서 성능이 높게 나타나는 문제를 최소화할 수 있었으며, 실제로 처음 접하는 입력에 대해서도 얼마나 안정적으로 분류가 가능한지를 보다 신뢰성 있게 평가할 수 있었다.

1.5) 전처리 결과 저장 및 관리

앞서 설명한 전처리와 데이터 분할 과정을 거친 뒤, 학습에 사용될 모든 데이터는 재사용성과 실험의 재현 가능성을 고려하여 파일 형태로 저장하였다. 전처리 결과물은 그림 7과 같이 train.npz, val.npz, test.npz 파일로 구성되며, 각 파일에는 로그 멜 스펙트로그램 특징 행렬과 이에 대응하는 클래스 레이블이 함께 포함되어 있다. 이러한 방식은 모델 학습 단계에서 매번 원본 오디오를 다시 처리할 필요 없이, 동일한 입력 데이터를 안정적으로 불러올 수 있도록 한다.

또한 데이터 정규화에 사용된 통계 정보는 그림 8과 같이 norm_stats.json 파일로 별도로 저장하였다. 해당 파일에는 훈련 데이터 기준으로 계산된 평균과 표준편차, 그리고 전체 클래스 목록이 포함되어 있으며, 이후 검증 및 테스트 데이터에도 동일한 정규화 기준을 적용할 수 있도록 설계되었다. 이를 통해 데이터 분할 간 스케일 차이로 인한 성능 왜곡을 방지하고, 실험 조건을 일관되게 유지할 수 있었다.

전처리 과정에서 생성된 각 세그먼트의 출처와 특성을 추적하기 위해 meta.csv 파일도 함께 구성하였다. 그림 9는 해당 메타데이터 파일의 일부를 나타낸 것으로, 각 세그먼트에 대해 원본 파일 경로, 클래스 ID와 클래스 이름, 세그먼트 시작 시간, 적용된 hop 크기, 그리고 train·validation·test 중 어느 데이터셋에 속하는지를 기록하고 있다. 이 메타데이터는 이후 특정 실험 결과를 분석할 때, 어떤 원본 파일과 전처리 조건에서 생성된 데이터인지 역추적하는 데 활용되었다.

이와 같은 저장 구조를 통해 본 프로젝트에서는 전처리 결과를 단순히 모델 입력으로 사용하는 데 그치지 않고, 실험 과정 전반을 체계적으로 관리할 수 있는 기반을 마련하였다. 이는 여러 실험에서 hop 크기나 모델 구조를 변경하며 비교 분석을 수행하는 과정에서도, 데이터 자체의 일관성과 신뢰성을 유지하는 데 중요한 역할을 하였다.

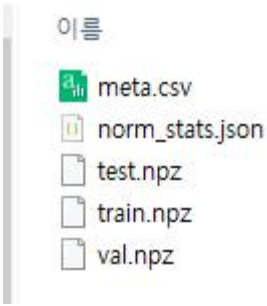


그림 3



그림 4

A	B	C	D	E	F
src_path	class_id	class_nam	start_sec	hop_sec	split
dataset_wav₩Chatting₩채팅 7.wav	0	Chatting	0	1	train
dataset_wav₩Chatting₩채팅 7.wav	0	Chatting	1	1	train
dataset_wav₩Chatting₩채팅 7.wav	0	Chatting	2	1	train
dataset_wav₩Chatting₩채팅 7.wav	0	Chatting	3	1	train
dataset_wav₩Chatting₩채팅 7.wav	0	Chatting	4	1	train
dataset_wav₩Chatting₩채팅 7.wav	0	Chatting	5	1	train
dataset_wav₩Chatting₩채팅 7.wav	0	Chatting	6	1	train
dataset_wav₩Chatting₩채팅 7.wav	0	Chatting	7	1	train
dataset_wav₩Chatting₩채팅 7.wav	0	Chatting	8	1	train
dataset_wav₩Chatting₩채팅 7.wav	0	Chatting	9	1	train

그림 5

1.6) 전처리 결과 시각화

앞서 설명한 전처리 과정을 모두 거친 후, 생성된 입력 데이터가 실제로 어떤 형태로 모델에 전달되는지를 확인하기 위해 학습 데이터 일부를 시각화하였다. 그림 6은 학습 시작 이전 단계에서 train 데이터셋에 포함된 로그 멜 스펙트로그램 샘플을 불러와 시각화한 결과로, 동일 클래스에 속한 세그먼트들을 나란히 비교할 수 있도록 구성하였다.

각 이미지는 하나의 세그먼트에 대해 생성된 로그 멜 스펙트로그램이며, 원래 모델 입력 형태는 (64, Time, 1) 구조이다. 이때 마지막 차원은 CNN 입력 형식에 맞추기 위해 추가된 채널 차원으로, 실제로는 색상 정보가 없는 단일 특성 맵을 의미한다. 시각화 과정에서는 해당 채널 차원이 의미를 갖지 않기 때문에, squeeze() 연산을 통해 마지막 채널을 제거하고 (64, Time) 형태로 변환한 뒤 2차원 이미지로 표현하였다.

이를 통해 멜 주파수 축 방향으로는 키보드 타건과 마우스 클릭과 같은 입력 이벤트가 주로 나타나는 특정 주파수 대역이 강조되어 있음을 확인할 수 있었으며, 시간 축 방향으로는 입력이 발생하는 시점에 따라 수직 패턴이 반복적으로 나타나는 것을 관찰할 수 있었다. 또한 동일한 클래스에 속한 세그먼트들은 전반적으로 유사한 시각적 특징을 보이면서도, 세부적인 패턴에서는 차이를 나타내어 데이터가 단순 복제 수준이 아님을 확인할 수 있었다.

이러한 시각화 결과를 통해 전처리 과정에서 생성된 로그 멜 스펙트로그램이 의도한 형태

로 구성되었으며, 모델이 학습에 활용할 수 있는 입력 구조로 정상적으로 준비되었음을 확인할 수 있었다.

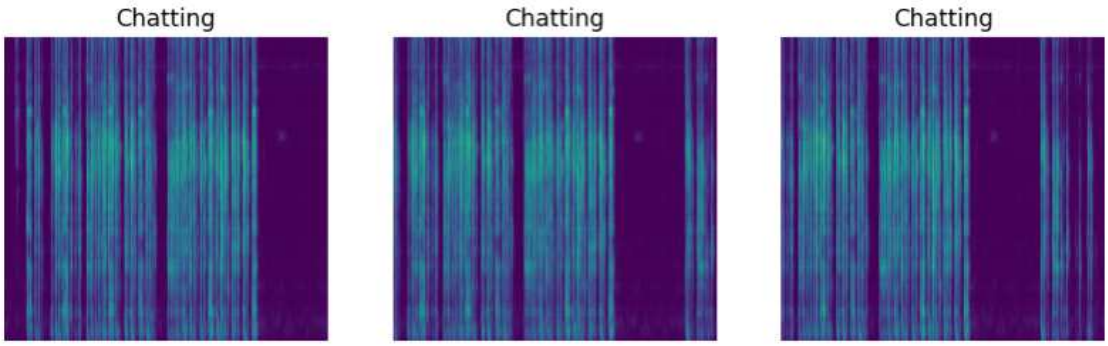


그림 6

정규화가 완료된 데이터는 학습, 검증, 테스트 세트로 구분되어 각각 npz 파일 형태로 저장되었다. 입력 데이터는 로그 멜 스펙트로그램을 기반으로 한 4차원 텐서 구조로 구성되었으며, 훈련 데이터는 (10455, 64, 1001, 1), 검증 데이터는 (2226, 64, 1001, 1), 테스트 데이터는 (1100, 64, 1001, 1)의 형태를 가진다.

각 텐서에서 첫 번째 차원은 데이터셋에 포함된 세그먼트의 개수를 의미하며, 이후 차원은 멜 필터 개수, 시간 축 길이, 채널 수를 순서대로 나타낸다. 세그먼트의 총 개수는 실험 단계별로 적용한 hop 크기에 따라 달라질 수 있으나, 본 보고서에서는 최종적으로 사용된 모델 학습 기준 데이터셋의 형태를 기준으로 제시하였다.

이와 같은 전처리 및 정규화 결과는 그림 8에 제시된 데이터 형태를 통해 확인할 수 있다. 해당 그림은 학습·검증·테스트 데이터셋의 크기와 입력 구조를 요약한 것으로, 로그 멜 스펙트로그램 기반 입력이 최종적으로 모델 학습에 적합한 형태로 구성되었음을 보여준다. 이러한 데이터 구성은 이후 실험 과정에서 모델 성능을 안정적으로 비교하고 분석할 수 있는 기반을 제공하였다.

```
훈련 데이터 형태: (10455, 64, 1001, 1)
검증 데이터 형태: (2226, 64, 1001, 1)
테스트 데이터 형태: (1100, 64, 1001, 1)
입력 모양: (64, 1001, 1)
```

그림 7

2. 모델 구조 및 설계

본 프로젝트에서는 로그 멜 스펙트로그램을 입력으로 사용하는 합성곱 신경망 기반 분류 모델을 설계하였다. 모델 구조는 초기 모델과 개선 모델로 구분하여 구성하였으며, 두 모델은 동일한 입력 데이터 형식을 사용하되 특징 추출 방식과 과적합 제어 전략 등에서 차이를 가진다.

2.1) 초기 모델 구조

초기 모델은 로그 멜 스펙트로그램을 입력으로 하여 합성곱 신경망 기반 분류가 실제로 가능한지 확인하기 위한 탐색적 구조로 설계되었다. 입력 데이터는 전처리 과정을 거쳐 생성된 로그 멜 스펙트로그램으로, 하나의 샘플은 세로 방향으로 64개의 멜 필터를 가지는 주파수 축과 시간 축으로 구성된 형태이며, 단일 채널을 갖는 구조이다. 이러한 입력 형태는 그림 1에 제시된 모델 입력 구조를 통해 확인할 수 있다.

그림 2는 초기 모델의 전체적인 신경망 구조를 나타낸다. 모델은 입력층 이후 세 개의 합성곱 블록으로 구성되었으며, 각 블록은 Conv2D, MaxPooling2D, BatchNormalization 계층을 순차적으로 포함하도록 설계하였다. 합성곱 계층의 채널 수는 16, 32, 64 순서로 점진적으로 증가하도록 설정하여, 낮은 수준의 시간·주파수 특징에서 점차 복잡한 특징으로 확장 학습이 가능하도록 의도하였다.

각 합성곱 연산 이후에는 MaxPooling2D를 적용하여 특징 맵의 크기를 줄이고 연산량을 감소시키는 동시에, 불필요한 세부 변동을 완화하도록 하였다. 이어서 BatchNormalization을 사용하여 각 배치마다 입력 분포를 정규화함으로써 학습 안정성을 높이고자 하였다. 이러한 블록 구성은 그림 3에 제시된 합성곱 블록 구조에서 확인할 수 있다.

합성곱 블록을 모두 통과한 특징 맵은 GlobalAveragePooling2D를 통해 하나의 벡터로 요약되었다. 이후 Dense 계층 64 유닛을 거쳐 최종 출력층에서 네 개 클래스에 대한 분류 결과를 생성하도록 구성하였다. 이 출력 구조는 그림 4에 나타난 분류기 부분에서 확인할 수 있다.

학습 과정에서는 Adam optimizer를 사용하였고, 정수 레이블을 그대로 사용하는 방식에 맞추어 SparseCategoricalCrossentropy 손실 함수를 적용하였다. 또한 ModelCheckpoint, EarlyStopping, ReduceLROnPlateau를 함께 사용하여 학습 과정에서의 과도한 갱신과 불필요한 반복 학습을 방지하고자 하였다. 미니 배치 학습은 tf.data.Dataset 기반으로 구성되었으며, 한 번의 학습 단계에서는 batch size 32에 해당하는 스펙트로그램 묶음이 모델 입력으로 전달되었다. 이 과정은 실제로 (32, 64, Time, 1) 형태의 텐서가 모델에 입력되는 구조임을 코드 실행 결과를 통해 확인하였다.

그러나 이러한 초기 모델은 설계 의도와 달리 실제 실험에서 만족스러운 성능을 보이지 못했다. 훈련 정확도는 비교적 빠르게 상승했지만, 검증 정확도는 큰 폭으로 변동하며 안정적으로 수렴하지 않는 모습을 보였다. 특히 GlobalAveragePooling2D를 사용한 특징 요약 방식과 제한적인 채널 수 구성만으로는 키보드와 마우스 입력처럼 짧고 반복적인 비음성 신호의 미세한 차이를 충분히 표현하기 어렵다는 한계가 드러났다. 결과적으로 이 초기 모델은 실험 파이프라인과 데이터 흐름을 검증하는 단계로서의 의미는 있었지만, 2회 실험 후 단순한 파라미터 조정만으로는 성능 개선이 어렵다고 판단되어 이후 모델 구조 자체를 전면적으로 재설계하게 되는 계기가 되었다.

```
print("--- 데이터셋 로딩 중 ---")
train_ds = load_dataset_from_npz("train", BATCH_SIZE)
val_ds = load_dataset_from_npz("val", BATCH_SIZE)
test_ds = load_dataset_from_npz("test", BATCH_SIZE)

# 데이터 형태 확인
for images, labels in train_ds.take(1):
    print(f"Input Shape: {images.shape}")  # 예: (32, 6
    print(f"Label Shape: {labels.shape}")  # 예: (32,
```

```
--- 데이터셋 로딩 중 ---
Input Shape: (32, 64, 1001, 1)
Label Shape: (32,)
```

그림 1

```
model = Sequential([
    # Input
    layers.Input(shape=(IMG_HEIGHT, None, 1)),

    # [Block 1]
    layers.Conv2D(16, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.BatchNormalization(),

    # [Block 2]
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.BatchNormalization(),

    # [Block 3]
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.BatchNormalization(),

    # [Global Pooling]
    layers.GlobalAveragePooling2D(),

    # [Classifier]
    layers.Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.001)),

    layers.Dropout(0.5),

    layers.Dense(num_classes)
])
```

그림 2

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
```

그림 3

```
# 1. 모델 저장 (그대로 유지)
checkpoint_cb = ModelCheckpoint(
    "best_model.keras",
    save_best_only=True,
    monitor="val_accuracy",
    mode="max",
    verbose=1
)

# 2. 조기 종료 (그대로 유지)
early_stopping_cb = EarlyStopping(
    monitor="val_loss",
    patience=7,
    restore_best_weights=True
)

reduce_lr_cb = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=3,
    min_lr=1e-6,
    verbose=1
)

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=30,
    callbacks=[checkpoint_cb, early_stopping_cb, reduce_lr_cb]
)
```

그림 4

2.2) 개선된 모델 구조

초기 모델 실험 결과, 제한적인 채널 수와 단순한 특징 요약 방식으로 인해 키보드·마우스 입력과 같이 짧고 반복적인 비음성 신호의 복잡한 패턴을 충분히 학습하지 못한다는 한계가 드러났다. 특히 검증 정확도의 변동 폭이 크고 과적합이 반복적으로 발생하면서, 기존 구조를 부분적으로 수정하는 수준이 아닌 모델 구조 전반의 개선이 필요하다고 판단하였다. 이에 따라 합성곱 채널 구성, 특징 요약 방식, 학습 안정화 기법을 중심으로 개선된 모델을 새롭게 설계하였다.

개선된 모델의 전체 구조는 그림 6에 제시된 코드와 같이 build_model() 함수로 정의되었다. 입력은 전처리 단계에서 생성된 로그-멜 스펙트로그램으로, 높이 방향은 멜 필터 개수 64, 시간 축은 가변 길이, 채널 수는 1로 구성된 형태이다. 이 입력은 합성곱 신경망을 통해 단계적으로 특징을 추출하도록 설계되었다.

모델의 합성곱 블록은 총 세 단계로 구성되어 있으며, 각 단계에서 채널 수를 32, 64, 128

로 점진적으로 확장하였다. 이는 입력 신호의 저수준 특징부터 고수준 특징까지 점차 풍부하게 표현하기 위한 설계이다. 각 블록은 Conv2D 계층과 ReLU 활성화 함수, 동일 패딩을 적용하여 공간 해상도를 유지한 상태로 특징을 추출하도록 하였으며, 이후 MaxPooling2D를 통해 시간 축을 강하게 압축하였다. 풀링 커널을 (2, 4)로 설정함으로써 주파수 축보다 시간 축을 더 크게 줄이도록 설계하였는데, 이는 키보드·마우스 입력음이 순간적으로 발생하는 이벤트 중심 신호라는 특성을 반영한 것이다. 각 블록 뒤에는 Dropout(0.25)를 적용하여 특정 시간 구간이나 패턴에 과도하게 의존하는 현상을 완화하였다.

합성곱 블록 이후에는 초기 모델과 달리 Flatten 계층을 사용하여 모든 특징 맵을 하나의 벡터로 펼친 뒤 분류기로 전달하였다. 이는 전역 평균 풀링을 사용하던 초기 모델과의 중요한 구조적 차이로, 시간·주파수 축 전반의 정보를 보다 직접적으로 활용하기 위한 선택이었다. 이후 Dense(128) 계층과 ReLU 활성화 함수를 통해 비선형 결합을 수행하였으며, Dropout(0.5)를 추가로 적용하여 완전 연결층에서의 과적합을 억제하였다. 최종 출력층에서는 Dense(NUM_CLASSES, activation='softmax')를 사용하여 네 개 클래스에 대한 확률 분포를 출력하도록 구성하였다.

학습 설정과 관련된 내용은 그림 7과 그림 5에 제시된 코드에서 확인할 수 있다. 최적화 알고리즘은 Adam을 사용하였으며, 학습률은 0.0001로 설정하여 너무 빠른 수렴으로 인한 불안정성을 방지하였다. 손실 함수는 categorical_crossentropy를 사용하였고, 정확도를 주요 평가 지표로 설정하였다. 학습 과정에서는 ModelCheckpoint를 통해 검증 정확도가 가장 높을 때의 모델을 저장하였으며, EarlyStopping을 적용해 검증 손실이 더 이상 개선되지 않을 경우 학습을 조기에 종료하도록 하였다. 또한 ReduceLROnPlateau를 사용해 검증 손실 정체 시 학습률을 단계적으로 감소시켜 보다 안정적인 수렴을 유도하였다. 클래스 불균형 문제를 완화하기 위해 class_weight를 계산하여 학습에 반영한 점 역시 개선된 모델의 중요한 특징이다.

데이터 입력 방식 측면에서는 그림 5에 나타난 것처럼, load_dataset_from_npz() 함수에서 배치 크기 32를 기준으로 미니배치가 구성된다. 즉, 한 번의 학습 스텝마다 (32, 64, Time, 1) 형태의 입력 텐서가 모델에 전달되며, 이에 대응하는 라벨 벡터가 함께 사용된다. 배치 크기는 모든 실험에서 동일하게 유지하였으며, 해당 설정에서도 손실과 정확도가 안정적으로 수렴하는 것을 확인하였다.

이와 같은 구조적·학습 전략적 개선을 통해, 개선된 모델은 초기 모델 대비 학습 과정이 훨씬 안정적으로 진행되었으며 검증 및 테스트 정확도 또한 큰 폭으로 향상되었다. 그 결과 본 프로젝트에서는 해당 개선 모델을 최종 모델로 채택하였다.

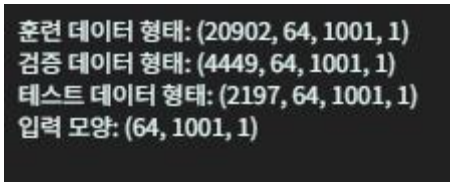


그림 5



그림 6



그림 7

정리하면, 본 프로젝트에서 설계한 두 가지 CNN 모델은 모두 로그-멜 스펙트로그램 기반 입력을 공통적으로 사용하며, 입력 데이터는 전처리 과정을 거쳐 (64, Time, 1) 형태의 2차원 특징 맵으로 구성된다. 여기서 64는 멜 주파수 필터 개수를 의미하며, Time 축은 10초 세그먼트에 대해 STFT와 멜 변환을 거치며 결정된 시간 프레임 길이이다. 마지막 채널 차원은 흑백 이미지와 동일한 단일 채널 구조를 의미한다.

학습 과정에서는 이 입력이 미니배치 단위로 묶여 모델에 전달되며, 실제 학습 시 입력 형태는 (batch_size, 64, Time, 1)로 구성된다. 이는 모델이 한 번의 학습 단계에서 여러 개의 로그-멜 스펙트로그램 샘플을 동시에 처리하도록 하기 위한 구조로, 본 프로젝트에서는 배치 크기를 32로 고정하여 사용하였다.

두 모델 모두 출력층에서는 클래스 개수에 해당하는 노드 수를 가진 Dense layer를 통해 최종 예측을 수행한다. 출력 값은 각 클래스에 대한 점수(logits) 또는 확률 분포로 해석되며, 이를 통해 Chatting, Game, Web Surfing, Writing 중 하나의 활동 클래스를 분류한다. 즉, 모델의 입출력 구조는 입력으로는 로그-멜 스펙트로그램, 출력으로는 활동 클래스 분류 결과라는 일관된 흐름을 가진다.

다만, 초기 모델과 개선 모델은 이 동일한 입출력 구조를 유지하면서도 특징을 추출하고 요약하는 내부 방식에서 뚜렷한 차이를 보인다. 초기 모델은 상대적으로 얕은 채널 구성과 전역 평균 풀링을 사용해 구조를 단순화한 반면, 개선 모델은 더 많은 합성곱 채널과 Flatten 기반의 특징 전개, 강한 드롭아웃 및 다양한 학습 제어 기법을 통해 복잡한 입력 패턴을 보다 풍부하게 학습하도록 설계되었다. 이 차이는 이후 실험 결과에서 모델 성능과 학습 안정성의 차이로 명확하게 나타났다.

3. 단계별 실험 설계 및 분석

프로젝트는 총 여섯 단계의 실험을 통해 모델의 성능을 점진적으로 개선하는 방식으로 진행되었다. 각 단계에서는 데이터 전처리 방식, hop 크기, 모델 구조와 규제 기법 등을 달리하여 어떤 요소가 성능 향상에 기여하는지 확인하였다.

3.1) 첫 번째 실험

첫 번째 실험에서는 클래스별 hop 크기를 Chatting과 Web Surfing은 0.5초, Game과 Writing은 1.5초로 설정하고, 초기 단계에서 설계한 기본적인 합성곱 신경망 구조를 그대로 적용하였다. 이 구조는 합성곱과 최대 풀링을 반복하는 단순한 형태였으며, 드롭아웃이나 배치 정규화와 같은 규제 기법이 포함되지 않아 학습 안정성이 낮은 편이었다.

학습 결과는 특정 클래스에 치우친 예측이 반복되는 등 분류 성능이 고르지 못했고, 훈련 정확도와 검증 정확도 사이의 차이가 크게 나타났다. 최종 테스트 정확도는 59.18퍼센트로 측정되었으며, 이는 모델이 데이터의 패턴을 충분히 학습하지 못했음을 보여준다. 아래 그림의 학습 곡선에서도 확인할 수 있듯이, 검증 정확도는 초반부터 급격히 하락하고 손실 값은 오히려 증가하는 양상을 보였다. 이러한 패턴은 모델이 복잡한 패턴을 표현할 충분한 용량을 갖추지 못했거나, 데이터 구성에서 중복과 불균형이 존재했을 가능성을 시사한다.

이러한 1차 시도 결과, 우선 hop 크기 같은 전처리 설정을 조정하면서 데이터 생성 방식을 바꾸어 보자고 생각을 하였다.

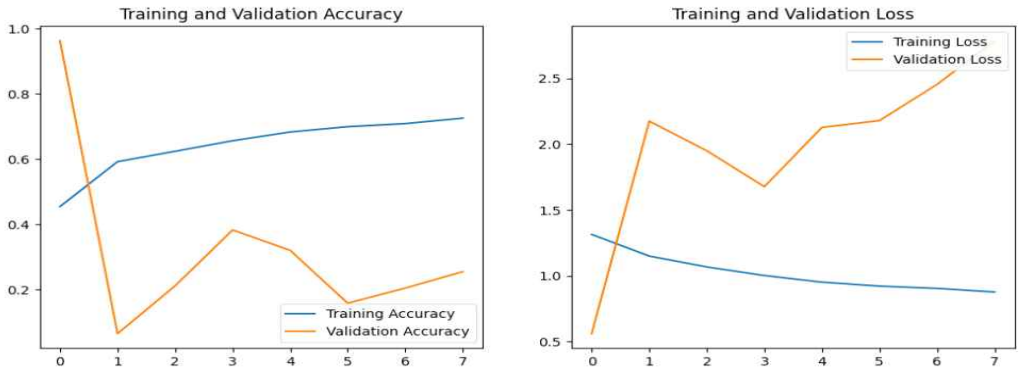


그림 1

```
★ 최고 성능 모델(best_model.keras) 로드 완료

[샘플 예측 테스트]
1/1 [==============================] - 0s 86ms/step
정답 클래스: Chatting
예측 클래스: Writing (46.81% 확신)
35/35 [==============================] - 2s 65ms/step - loss: 1.3461 - accuracy: 0.5918

최종 테스트 정확도: 59.18%
```

그림 2

3.2) 두 번째 실험

두 번째 실험에서는 전처리 단계가 성능에 미치는 영향을 확인하기 위해 hop 크기만 조정하였다. Chatting과 Web Surfing은 1초, Game과 Writing은 3초로 설정하여 세그먼트 중복을 줄이고 데이터 다양성을 확보하려는 목적이었다. 모델 구조는 1차 시도와 동일한 초기 CNN 구조를 그대로 사용하였다.

그러나 hop 크기 변경만으로 성능 개선을 이루기에는 한계가 있었다. 아래 그래프를 보면 검증 정확도는 첫 번째 epoch 이후 거의 0 근처에서 머무르며 상승하지 않았고, 이는 모델이 검증 데이터의 패턴을 전혀 일반화하지 못했음을 의미한다. 반면 훈련 정확도는 완만하게 증가하는 모습을 보였는데, 이는 학습 데이터에만 맞춰지는 편향된 학습, 즉 과적합이 나타난 것으로 해석할 수 있다.

검증 손실 역시 전체 epoch 동안 지속적으로 증가하는데, 특히 2~4 epoch 구간에서 급격히 치솟는 양상이 나타났다. 이는 모델이 훈련을 거듭할수록 검증 데이터와 멀어지는 방향으로 파라미터가 업데이트되고 있음을 보여주며, 단순한 전처리 변화로는 이런 문제를 해결하기 어려움을 알았다.

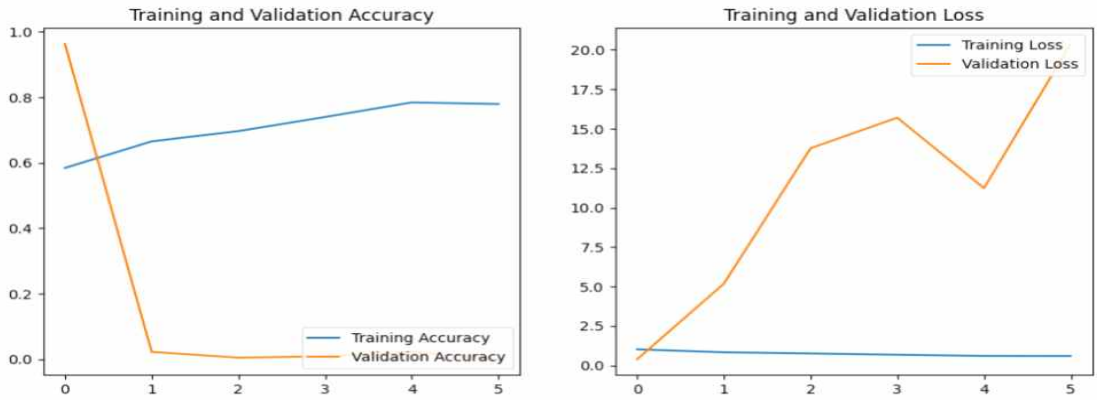


그림 3

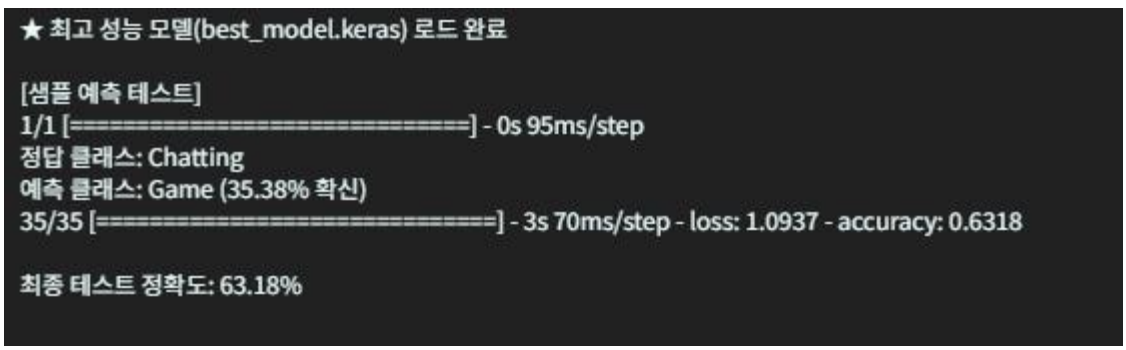


그림 4

최종 테스트 정확도는 63.18퍼센트로 측정되었으며, 이는 1차 시도 대비 소폭 개선된 수치이지만 여전히 실질적인 분류 성능으로 보기에는 부족한 수준이었다. 이러한 결과를 바탕으

로 hop 크기 변경만으로는 성능 안정화에 한계가 있으며, 모델 구조 개선이 필요하다는 판단을 내리게 되었다.

3.3) 세 번째 실험

세 번째 실험부터는 모델 구조 자체를 개선할 필요성이 있다고 판단하여, 임베디드 수업 실습 시간에 교수님이 주신 예제 코드를 참고해 전체적인 모델 구성 방식을 다시 설계하였다. 이 예제는 합성곱 신경망의 블록 구조, 활성화 함수의 배치 방식, 드롭아웃 활용 방법 등 기본적인 설계 원칙이 안정적으로 반영된 구조였기 때문에, 앞선 시도에서 나타난 불안정한 학습 패턴을 해결하는 데 적합하다고 생각하였다. 제공된 코드가 텐서플로우 1.5 버전을 기반으로 작성되어 있었기 때문에, 실험 환경의 호환성을 맞추기 위해 텐서플로우 버전을 다시 1.5로 낮추어 실험을 진행하였다.

3차 시도에서는 모델 구조의 안정화뿐 아니라, 클래스 간 데이터 불균형을 보정하기 위해 class_weight를 도입한 점도 중요한 변화였다. Chatting과 Web Surfing은 데이터량이 상대적으로 적고, Game과 Writing은 더 많이 존재하는 구조였기 때문에 단순한 학습만으로는 특정 클래스에 대한 과한 편향이 발생할 위험이 있었다. 모델 학습 시 각 클래스에 가중치를 부여하도록 설정함으로써, 학습 과정에서 적은 데이터의 클래스도 충분한 비중으로 학습되도록 조정하였다. 이는 앞선 두 실험에서 나타났던 특정 클래스 오예측 문제를 완화하는 데 도움이 되었다.

아래 그래프를 보면, 훈련 정확도는 초기 몇 epoch에서 급격히 상승한 뒤 98퍼센트 이상을 유지하며 안정적으로 수렴하는 모습을 보인다. 검증 정확도 역시 약 99퍼센트 수준을 지속적으로 유지하고 있어, 모델이 과도하게 치우치지 않고 전체 패턴을 균형 있게 학습하고 있음을 확인할 수 있다. 손실 그래프에서도 같은 경향이 나타난다. 훈련 손실과 검증 손실 모두 매우 낮은 값으로 수렴하며, 두 값의 차이가 크지 않아 과적합 문제를 크게 완화하는 데 성공한 것으로 판단된다. 이는 1차와 2차 시도에서 보였던 검증 손실의 급격한 증가와 정확도의 붕괴 현상과는 뚜렷하게 대비된다.

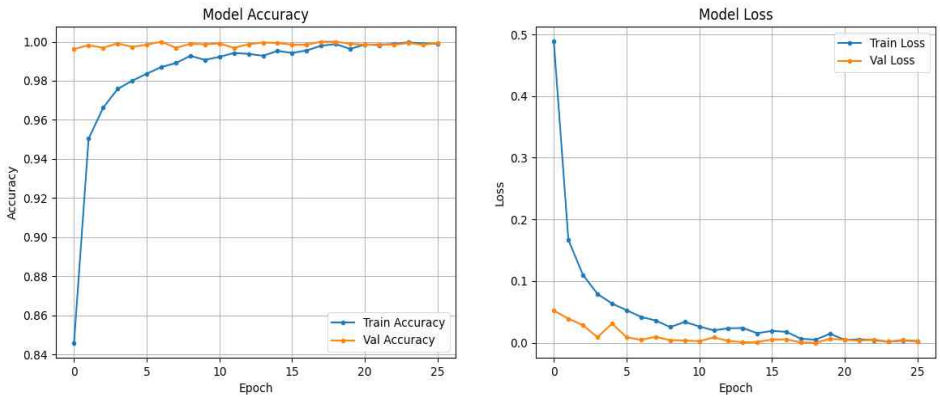


그림 5

모델 구조와 가중치 적용 방식을 포함해 학습 전략이 안정화된 것을 확인한 이후, 테스트 데이터를 이용해 별도의 검증 코드를 통해 모델을 다시 평가하였다. 그 결과 최종 테스트 정확도는 약 81퍼센트로 측정되었다. 이는 1차 시도 59퍼센트, 2차 시도 63퍼센트에 비해 분류 능력이 크게 향상된 수치로, 구조적 설계 개선과 데이터 불균형 처리의 효과가 직접적으로 반영된 결과라고 할 수 있다.

3차 시도는 모델 구조의 안정성과 불균형 처리 전략이 성능에 미치는 영향이 매우 크다는 점을 분명히 확인한 단계였다. 이러한 안정적인 기반이 마련되었기 때문에, 이후 실험에서는 아까 1, 2차 실험에서 변경한 hop 크기 조정이 실제 모델 성능에 어떤 영향을 주는지를 분석해 볼 수 있다고 생각하였다.

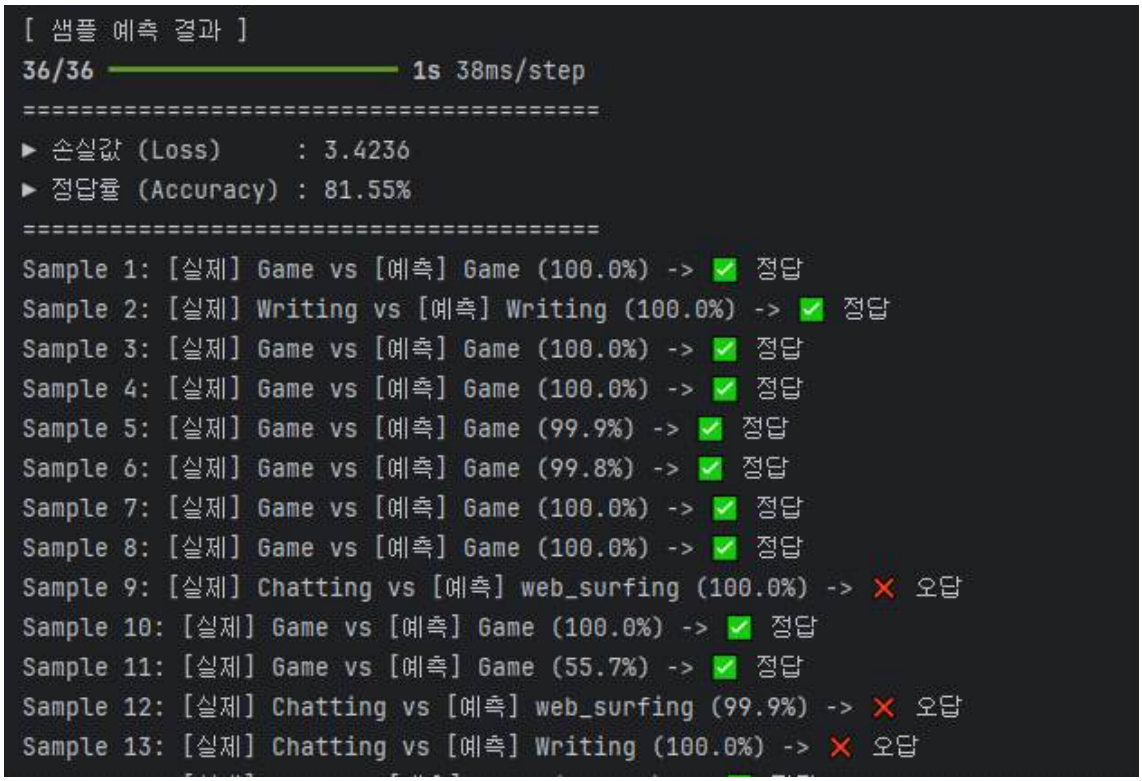


그림 6

3.4) 네 번째 실험

네 번째 실험에서는 3차 실험에서 안정화된 모델 구조를 그대로 유지한 상태에서, 데이터 생성 과정의 핵심 요소인 hop 크기만을 변경하여 성능 변화를 확인하였다. 이전 실험에서는 hop을 1초와 3초로 설정하였으나, 네 번째 실험에서는 hop을 0.5초와 1.5초 조합으로 조정하여 세그먼트 밀도를 높이고, 클래스 간 데이터 균형이 보다 자연스럽게 형성되도록 구성하였다.

아래 그래프를 보면, 훈련 정확도는 초반부터 빠르게 상승하여 전반적으로 높은 수준을 유지하고 있으며, 검증 정확도 역시 안정적으로 수렴하는 양상을 보인다. 그래프의 형태는 3차 실험과 유사해 보이지만, 세부적으로는 초기 검증 정확도가 더 빠르게 안정되는 모습을 확

인할 수 있다. 이는 hop 크기 조정으로 인해 생성된 데이터의 다양성이 증가하면서 모델이 초반부터 더 일관된 패턴을 학습할 수 있었음을 의미한다.

손실 그래프에서도 같은 경향이 나타난다. 검증 손실은 전체 학습 과정에서 크게 흔들리지 않고 낮은 값을 유지하고 있으며, 훈련 손실과 거의 비슷한 수준으로 수렴하고 있다. 이러한 결과는 데이터 구성에서 중복이나 편향이 완화되면서 과적합 위험이 줄어들었음을 나타낸다.

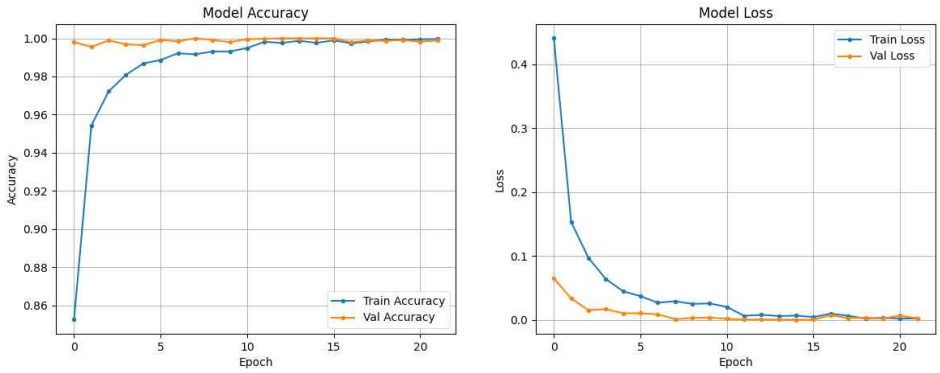


그림 7

추가로 진행한 테스트 단계에서는 최종 정확도가 약 85퍼센트로 측정되었다. 이는 동일한 모델 구조를 사용한 3차 실험보다 높은 수치이며, 모델 구조의 변화 없이 hop 조정만으로도 성능이 개선될 수 있음을 보여준다. 특히 1차와 2차 시도에서 계속 문제가 되었던 검증 정확도의 급락 현상이 더 이상 나타나지 않는다는 점에서, 데이터 전처리 방식이 모델 성능에 중요한 영향을 미친다는 것을 확인할 수 있었다.

```
[ 샘플 예측 결과 ]
69/69 ━━━━━━━━━━━━━━━━━━━━ 3s 36ms/step
========================================
▶ 손실값 (Loss)     : 1.3165
▶ 정답률 (Accuracy) : 84.71%
========================================
Sample 1: [실제] Game vs [예측] Game (99.9%) -> ✅ 정답
Sample 2: [실제] Game vs [예측] Game (100.0%) -> ✅ 정답
Sample 3: [실제] Game vs [예측] Game (100.0%) -> ✅ 정답
Sample 4: [실제] Game vs [예측] Game (100.0%) -> ✅ 정답
Sample 5: [실제] Game vs [예측] Game (100.0%) -> ✅ 정답
Sample 6: [실제] Game vs [예측] Game (79.4%) -> ✅ 정답
Sample 7: [실제] Game vs [예측] Game (99.9%) -> ✅ 정답
Sample 8: [실제] Game vs [예측] Game (100.0%) -> ✅ 정답
Sample 9: [실제] Chatting vs [예측] Writing (100.0%) -> ❌ 오답
Sample 10: [실제] Game vs [예측] Chatting (72.7%) -> ❌ 오답
Sample 11: [실제] Game vs [예측] web_surfing (96.0%) -> ❌ 오답
Sample 12: [실제] Game vs [예측] Game (89.0%) -> ✅ 정답
Sample 13: [실제] Game vs [예측] Game (100.0%) -> ✅ 정답
Sample 14: [실제] Game vs [예측] Game (100.0%) -> ✅ 정답
Sample 15: [실제] Game vs [예측] Game (99.4%) -> ✅ 정답
Sample 16: [실제] Game vs [예측] web_surfing (94.8%) -> ❌ 오답
Sample 17: [실제] Game vs [예측] web_surfing (93.5%) -> ❌ 오답
```

그림 8

3.5) 다섯 번째 실험

다섯 번째 실험에서는 이전 단계에서 사용한 모델 구조를 동일하게 유지한 채, hop 크기를 다시 한 번 조정하여 데이터 분할 방식이 성능에 미치는 영향을 확인하고자 하였다. 이번 실험에서는 hop을 0.5초와 3초 조합으로 설정하여, 앞선 실험들보다 시간 축 압축을 더 강하게 적용하는 형태의 데이터를 구성하였다. 이는 짧은 단위의 패턴을 더 촘촘하게 반영하면서도 장시간 세그먼트 간 중복은 줄이는 방향이 될 것이라 판단한 시도였다.

아래 그래프를 보면, 훈련 정확도는 비교적 빠르게 상승하여 전체적으로 높은 수준을 유지하고 있으며, 검증 정확도 또한 안정적인 흐름을 보이고 있다. 그러나 그래프 전반의 형태는 4차 실험과 크게 다르지 않고, 초기 몇 epoch 이후부터는 거의 동일한 수준에서 수렴하는 모습을 확인할 수 있다. 손실 그래프 역시 훈련 손실과 검증 손실 모두 낮은 값으로 수렴하고는 있지만, 4차 실험에서 나타난 패턴과 유사하여 hop 조정의 누적 효과가 뚜렷하게 나타난다고 보기는 어려웠다.

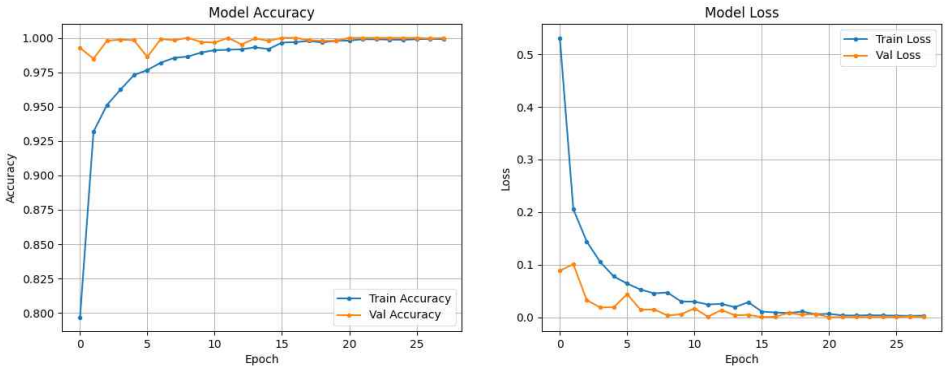


그림 9

추가 테스트 단계에서는 최종 정확도가 약 88퍼센트로 측정되었다. 이는 4차 실험보다 소폭 향상된 결과이지만, hop 크기만을 변경한 점을 고려하면 성능 차이가 충분히 유의미한 수준이라고 보기는 어려웠다. 특히 모델 구조와 규제 기법이 동일한 상황에서 hop 변경만으로 데이터를 다시 생성할 경우, 성능이 크게 달라지지 않는다는 경향이 반복적으로 확인되었다.

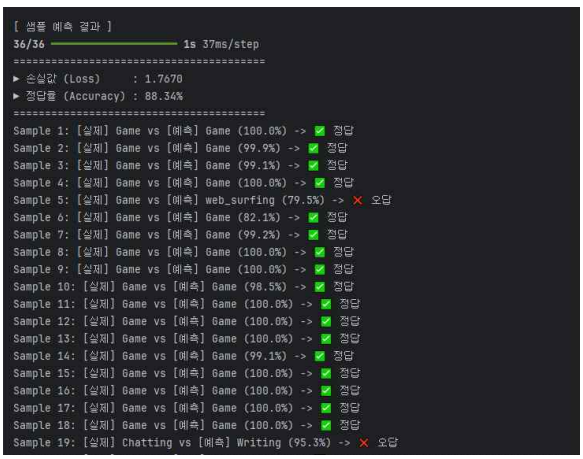


그림 10

3.6) 마지막 실험

여섯 번째 실험에서는 텐서플로우 버전을 다시 최신 버전으로 올려 환경을 구성한 뒤, 앞선 실험에서 성능이 안정적으로 확보된 모델 구조를 유지한 상태에서 학습을 진행하였다. 이 실험은 모델 구조 자체의 효과가 프레임워크 버전에 따라 어떻게 달라지는지 확인하기 위한 목적도 있었으며, 결과적으로 가장 높은 정확도를 기록한 실험에 해당한다.

아래 그래프를 보면, 훈련 정확도는 비교적 빠르게 상승하면서 이전과 비슷한 수준에 도달하고 있으나, 전체적인 곡선의 안정성은 4차와 5차 실험에 비해 다소 흔들리는 모습을 보인다. 특히 epoch 중반에서 훈련 정확도가 크게 요동치는 구간이 나타나며, 검증 정확도 또한 일정한 흐름을 유지하기보다는 특정 지점에서 상승과 하락이 반복되는 양상을 보이고 있다. 손실 그래프에서도 이러한 불안정성이 확인되며, 검증 손실이 일정한 감소 추세를 유지하지 못하고 중간에 다시 증가하는 구간이 나타난다.

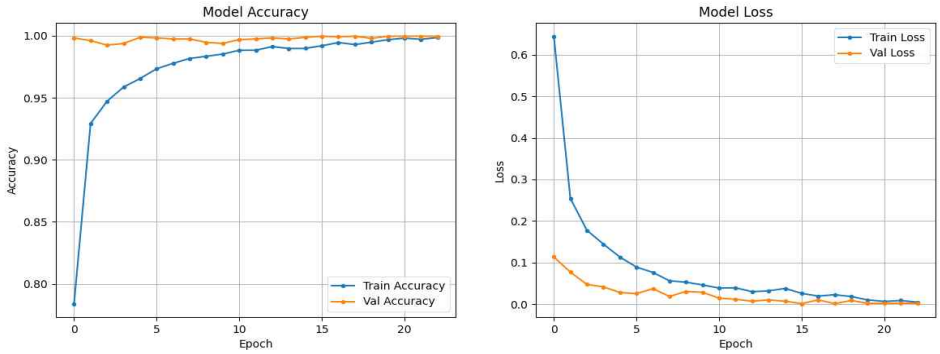


그림 11

추가적인 테스트 단계를 통해 측정한 최종 정확도는 약 91퍼센트로 확인되었다. 이는 5차 실험보다 소폭 향상된 수치이다. 그러나 그래프의 불안정한 형태와 성능 향상 폭이 크지 않다는 점을 고려하면, 프레임워크 버전 변경이 실질적인 성능 개선을 가져왔다고 보기는 어렵다고 생각된다. 또한, 버전이 바뀌며 기존에 사용하던 검증 코드를 사용하지 못해 이를 변경한 것이 미약하나마 영향이 있을것이라 판단하였다.



그림 12

위와 같은 결과를 종합해 보면, 동일한 모델 구조에서 텐서플로우 버전 변경은 성능의 근본적인 변화를 유도하기 어려웠으며, 이전 실험들에서 확인된 성능 개선 요소 역시 주로 모델 구조 안정화와 전처리 방식에 의한 것임을 다시 확인할 수 있었다. 이로써 hop 조정과 버전 변경까지 포함한 주요 실험 요소를 모두 검토한 것으로 판단하였으며, 더 이상의 반복 실험은 의미 있는 성능 향상을 가져오기 어렵다고 결론을 내리고 실험을 종료하였다.

4. 최종 성능 평가

여섯 단계의 실험을 통해 전처리 파이프라인과 CNN 모델 구조를 반복적으로 조정한 결과, 최종 모델은 테스트 기준 약 90퍼센트 수준의 분류 정확도를 기록하였다. 이는 1차와 2차 실험에서 확인된 50퍼센트대 후반과 60퍼센트대 초반 성능과 비교해 큰 폭으로 향상된 값이며, 전처리 안정화와 모델 구조 개선이 성능에 직접적인 영향을 주었음을 보여준다. 특히 3차 실험 이후에는 모델이 일정 수준 이상의 성능을 안정적으로 확보하기 시작했고, 이후 실험에서는 hop 설정 조합과 학습 환경 변화가 성능에 미치는 영향을 비교하며 최종 구성을 확정하는 방식으로 진행되었다.

최종 모델은 로그 멜 스펙트로그램을 입력으로 받아 합성곱 계층을 통해 시간·주파수 영역의 특징을 단계적으로 추출한 뒤, 드롭아웃을 포함한 분류기를 통해 네 개 활동 클래스를 예측한다. 학습 과정에서는 클래스 불균형 문제를 완화하기 위해 class_weight를 적용했고, 검증 성능을 기준으로 최적 모델을 저장하는 ModelCheckpoint, 과도한 학습을 방지하는 EarlyStopping, 검증 손실 정체 시 학습률을 낮추는 ReduceLROnPlateau를 함께 사용하여 학습의 안정성을 확보했다. 이 구성은 초기 모델에서 나타났던 검증 정확도의 급격한 붕괴와 손실 증가 양상을 완화하는 데 중요한 역할을 했다.

학습 이후에는 최종 모델의 출력이 실제로 안정적인지 확인하기 위해, 테스트 세트 샘플에 대해 예측 확률 분포를 직접 확인하는 검증을 수행하였다. 이 과정에서 모델이 특정 클래스에만 치우친 출력을 반복하는지, 혹은 특정 입력 조건에서 예측이 크게 흔들리는지 점검했으며, 전체적으로는 네 개 클래스에 대해 비교적 일관된 예측 경향을 보이는 것을 확인하였다. 다만 키보드 입력 패턴이 서로 유사하거나 마우스 이벤트가 적게 포함된 세그먼트처럼 구분 근거가 약한 입력에서는 오분류 사례가 발생했다. 이는 클래스별 데이터 길이 차이와 세그먼트 내부 이벤트 밀도의 차이, 그리고 입력 환경 변화에 따른 신호 변동이 완전히 제거되기 어렵다는 본 과제 특성에서 기인한 한계로 볼 수 있다.

종합하면, 최종 모델이 기록한 약 90퍼센트 수준의 성능은 프로젝트의 데이터 규모와 입력 조건을 고려했을 때 의미 있는 결과라고 판단된다. 무엇보다 초기 시도에서는 전처리 설정과 모델 구조의 미성숙으로 인해 학습과 검증의 흐름이 불안정했으나, 이후 전처리 체계화, 데이터 누수 방지 분할 전략 유지, class_weight와 콜백 기반 학습 제어, 그리고 모델 구조 전면 개선을 거치며 성능이 단계적으로 향상되었다는 점이 본 프로젝트의 핵심 성과이다.

5. 한계점 및 향후 개선 방향

본 프로젝트는 여러 단계의 실험을 통해 전처리 방식과 모델 구조를 점진적으로 개선하며 성능을 향상시켰다. 그러나 실험을 진행하는 과정에서 데이터 조건과 문제 특성으로 인한 한계 역시 분명하게 드러났다.

가장 큰 한계는 전체 데이터 양이 충분하지 않았다는 점이다. 녹음된 활동 구간이 길지 않았기 때문에, 세그먼트 수를 늘리기 위해 hop 간격을 조정하는 방식을 사용하였다. 이 방법은 학습에 필요한 샘플 수를 확보하는 데에는 효과적이었지만, 실제로는 동일한 원본 파일에서 파생된 유사한 데이터가 증가하는 구조였다. 따라서 데이터의 절대적인 다양성을 크게 확장하는 데에는 한계가 있었고, 모델의 일반화 성능을 근본적으로 끌어올리는 데에는 제약으로 작용하였다.

또한 활동별 데이터 분포가 균등하지 않았다는 점도 중요한 한계로 나타났다. Game과 Writing 클래스에 비해 Chatting과 Web Surfing 클래스는 데이터 양이 적고, 입력 패턴 또한 짧고 불규칙한 특성을 가진다. 이로 인해 일부 클래스는 특징을 안정적으로 학습하기 어려웠으며, 실험 과정에서도 특정 클래스에서 반복적인 오분류가 발생하는 경향이 확인되었다. class weight를 적용하여 데이터 불균형을 완화하고자 하였으나, 이는 학습 과정에서의 보정 역할에 가까웠고, 데이터 자체의 불균형을 완전히 해소하기에는 부족하였다.

녹음 환경 측면에서도 한계가 존재한다. 본 프로젝트에서는 키보드와 마우스 입력 패턴 자체에 집중하기 위해 배경 소음을 최대한 최소화한 상태에서 데이터를 수집하였다. 이로 인해 모델은 비교적 깨끗한 입력 신호를 기준으로 학습되었으며, 실제 환경에서 발생할 수 있는 다양한 배경 소음 상황은 충분히 반영되지 못했다. 이는 모델의 실제 적용 가능성을 제한하는 요소로 작용할 수 있다.

다만 이러한 한계는 향후 개선 방향을 함께 제시한다. 우선 데이터 수집량을 늘리고, 서로 다른 공간과 장비, 다양한 소음 조건에서 녹음을 진행한다면 모델이 학습할 수 있는 입력 패턴의 폭을 크게 확장할 수 있을 것이다. 또한 멜 스펙트로그램 외에 추가적인 음향 특징을 함께 사용하거나, 시간적 정보를 더 강하게 반영할 수 있는 모델 구조를 적용한다면 클래스 간 미세한 차이를 보다 안정적으로 학습할 가능성도 있다.

이번 프로젝트는 제한된 조건 속에서도 전처리 설계와 모델 구조 개선이 성능 향상에 얼마나 중요한 역할을 하는지를 확인한 과정이었다. 동시에 데이터 양과 다양성이 딥러닝 기반 분류 성능에 미치는 영향이 매우 크다는 점을 직접 체감한 경험이기도 하다. 이러한 한계를 바탕으로 향후 데이터 확장과 모델 구조 개선이 이루어진다면, 키보드·마우스 입력 기반 활동 인식 모델은 보다 높은 정확도와 실제 활용 가능성을 갖출 수 있을 것으로 기대된다.

Ⅲ. 결론

1. 고찰

프로젝트를 시작할 당시 가장 큰 우려는 키보드 입력음과 마우스 클릭음처럼 짧고 비음성적인 소리가 과연 학습을 통해 안정적으로 구별될 수 있을지에 대한 부분이었다. 실제로 사람의 귀로 들었을 때 네 가지 활동 간의 차이는 매우 미묘하게 느껴졌으며, 이러한 입력 패턴을 모델이 의미 있는 기준으로 구분할 수 있을지 확신하기 어려웠다. 특히 음성 인식과 달리 명확한 발화 구조나 단어 단위가 존재하지 않는다는 점에서, 분류 문제로 접근하는 것 자체가 가능할지에 대한 의문이 컸다.

초기 단계에서는 음성 파일을 그대로 모델에 입력해야 한다는 막연한 생각만 가지고 있었고, 어떤 형태의 입력이 학습에 적합한지조차 명확히 이해하지 못한 상태였다. 이로 인해 모델 구조를 설계하는 과정뿐 아니라, 전처리 단계 전반에 대해서도 방향을 잡기 어려웠다. 데이터의 무작위성과 클래스 간 불균형 문제 역시 처음에는 해결이 쉽지 않을 것이라는 부담으로 다가왔다.

그러나 전처리 과정에서 로그 멜 스펙트로그램이라는 표현 방식을 적용할 수 있다는 점을 이해하면서 상황이 달라졌다. 음향 신호를 시간과 주파수 축을 갖는 2차원 수치 배열로 변환함으로써, 수업 시간에 배운 합성곱 신경망 구조를 그대로 활용할 수 있다는 점이 큰 전환점이 되었다. 이때부터 음향 분류 문제를 완전히 새로운 영역으로 보기보다는, 이미지 분류 문제와 유사한 구조로 접근할 수 있다는 인식이 생겼고, 모델 설계에 대한 막연한 두려움도 상당 부분 해소되었다.

이후 세그먼트 분할 방식과 hop 크기를 조정하고, 데이터 분할 과정에서 발생할 수 있는 데이터 누수 문제를 직접 고민하며 전처리 파이프라인을 점차 정교화해 나갔다. 특히 동일한 원본 파일에서 생성된 세그먼트가 학습과 평가 데이터에 동시에 포함될 수 있다는 점을 인지하고, 이를 방지하기 위해 파일 단위 분할 방식을 적용한 경험은 이론으로만 알고 있던 데이터 누수 개념을 실제 문제로 체감하게 만든 계기였다. 이러한 과정은 단순히 성능을 높이기 위한 조치라기보다는, 모델 평가의 신뢰성을 어떻게 확보해야 하는지를 고민하게 만든 중요한 경험이었다.

여러 단계의 실험을 거치면서 특히 인상 깊었던 점은, 사람이 청각적으로는 거의 구분하지 못하는 패턴이라 하더라도 신경망은 반복적인 학습을 통해 점차 안정적인 분류 기준을 형성해 나간다는 사실이었다. 세 번째 실험 이후부터 훈련과 검증 정확도가 함께 안정적으로 수렴하는 모습을 보이기 시작했고, 이를 통해 초기 단계에서 가졌던 정말로 구분이 가능할까라는 의문에 대해 실험 결과로 답을 얻을 수 있었다. 이는 단순히 정확도 수치 이상의 의미를 가지며, 머신러닝 모델이 데이터를 통해 패턴을 학습하는 과정을 직접 확인했다는 점에서 의미 있는 경험이었다.

최종적으로 약 90퍼센트 이상의 테스트 정확도를 기록하면서, 제한된 데이터와 비교적 단순한 모델 구조에서도 일정 수준의 분류 성능을 달성할 수 있음을 확인하였다. 동시에 모델

학습 과정에서 출력되는 정확도와, 학습이 종료된 이후 별도로 작성한 검증 코드로 평가했을 때의 정확도 사이에 차이가 발생한다는 점도 확인할 수 있었다. 이 차이는 학습 중 평가가 특정 데이터 흐름이나 배치 구성에 부분적으로 의존하고 있을 가능성, 혹은 평가 방식의 차이에서 비롯된 결과일 수 있으며, 모델 성능을 해석할 때 단순한 학습 로그만으로 판단해서는 안 된다는 의문을 남겼다. 이 부분은 향후 학습과 평가 과정의 차이를 더 깊이 이해해야 할 필요성을 느끼게 한 지점이었다.

이번 프로젝트를 통해 머신러닝이라는 과목에 대해서도 이전보다 훨씬 큰 흥미를 갖게 되었다. 이론 수업에서 개념적으로만 접했던 전처리, 정규화, 데이터 누수, 과적합, 규제 기법과 같은 요소들을 실제 코드와 실험 결과를 통해 직접 고민해보면서, 단순히 내용을 아는 것과 이해하는 것 사이에는 분명한 차이가 있다는 점을 체감하였다. 특히 실험이 예상과 다르게 흘러갈 때 그 원인을 추적하고, 구조와 데이터를 하나씩 점검해 나가는 과정은 이론 수업에서 이해하지 못했던 부분들을 스스로 정리하는 계기가 되었다.

결과적으로 본 프로젝트는 단순히 하나의 분류 모델을 구현하는 데서 그치지 않고, 머신러닝 문제를 어떻게 정의하고, 데이터를 어떻게 다루며, 실험 결과를 어떻게 해석해야 하는지에 대해 직접 경험해볼 수 있었던 과정이었다. 초기의 불안과 막막함이 반복적인 실험과 개선을 통해 점차 확신으로 바뀌어 가는 과정을 거치면서, 머신러닝이라는 분야에 대해 더 깊이 이해하고 계속 탐구해보고 싶다는 동기를 얻을 수 있었다는 점에서, 본 프로젝트는 개인적으로도 의미 있는 학습 경험이었다.

참고자료

1. [Hands-On] 소리를 이미지 정보로 변환하는 다양한 기술들

(medium, https://medium.com/@hugmanskj/hands-on-%EC%86%8C%EB%A6%AC%EB%A5%BC-%EC%9D%B4%EB%AF%B8%EC%A7%80-%EC%A0%95%EB%B3%B4%EB%A1%9C-%EB%B3%80%ED%99%98%ED%95%98%EB%8A%94-%EB%8B%A4%EC%96%91%ED%95%9C-%EA%B8%B0%EC%88%A0%EB%93%A4-4b1b915e0ee5)

2. 소리 데이터를 활용한 딥러닝- Mel Spectrogram

(티스토리 블로그(걸어서 개발 한바퀴), https://walkaroundthedevelop.tistory.com/129)

3. 로그-멜 스펙트로그램(Log-Mel Spectrogram)

(티스토리 블로그(Groove.Log),https://hong3.org/387)

4. 과적합(Overfitting)을 막는 방법들

(위키 독스, https://wikidocs.net/61374)